CEE6501 — Lecture 11.3

text

Learning Objectives

By the end of this lecture, you will be able to:

- explain

Agenda

- Part 1 —

Part 1 - Environment Setup

In [474]:

```
# -------------------------
# Import Libraries and Helper Functions
# -------------------------
# This slide imports the required Python libraries and adds the
# course helper-code directory to the Python path.

import sys
import os
import numpy as np
import pandas as pd
```

In [475]:

```
# Get current notebook directory
current_dir = os.getcwd()

# Go up two levels, then into Code/L8
helpers_path = os.path.abspath(
    os.path.join(current_dir, "..", "..", "Code", "L8")
)

sys.path.append(helpers_path)

# Go up two levels, then into Code/L11
helpers_path = os.path.abspath(
    os.path.join(current_dir, "..", "..", "Code", "L11")
)

sys.path.append(helpers_path)
```

In [476]:

```
import numpy as np

from helper_funcs_dsm import (
    assemble_global_stiffness_and_fef,
    partition_system,
    assemble_global_displacements,
    assemble_global_forces
)
from helper_funcs_output import (
    print_element,
    print_dsm_results,
    print_matrix_scaled,
    print_vector_scaled
)
from helper_funcs_truss import (
    cosines_from_degrees,
    geometry_from_nodes,
    k_global_2d_truss_elastic,
    k_global_2d_truss_geometric
)
```

Part 2 - Problem Definition

In [477]:

```
# -------------------------
# Problem data (hard-coded)
# -------------------------
E = 70e6          # kPa, kN/m^2
A = 645.2e-6       # m^2

nb_1 = [0, 0]
ne_1 = [4, 3]

nb_2 = [8, 0]
ne_2 = [4, 3]

nb_3 = [0, 0]
ne_3 = [8, 0]
```

In [478]:

```
# -------------------------
# Problem data (hard-coded)
# -------------------------
n_nodes = 3
ndof_per_node = 2

# Global system size (still using 1-based maps here)
ndof = n_nodes * ndof_per_node

# Initialize Applied Load
F_global = np.zeros(ndof, dtype=float)

# Initialize Global Displacement
u_global = np.zeros(ndof, dtype=float)
```

In [479]:

```
# -------------------------
# User-Specified Boundary Conditions
# -------------------------

# Applied Load (0-indexed)
F_global[4-1] = -2000 #kN*m

# Restrained DOFs
dof_restrained_1based = np.array([1, 2, 6], dtype=int)

print(F_global)
```

```
[    0.     0.     0. -2000.     0.     0.]
```

In [480]:

```
# -------------------------
# User-Specified DOF Mapping
# -------------------------

# Mapping
m1 = np.array([1,2,3,4])
m2 = np.array([5,6,3,4])
m3 = np.array([1,2,5,6])

map_list = [m1, m2, m3]   # 1-based DOF maps
```

Part 3a - Iteration 0, Setup

In [481]:

```
# -------------------------
# Member 1
# -------------------------
L1, cx1, cy1, T1 = geometry_from_nodes(nb_1, ne_1)

#Start of the problem, there is no deformation in the member yet
Lbar_1 = np.copy(L1)

v1 = Lbar_1 - L1
q1 = E * A * v1 / L1

k1_e = k_global_2d_truss_elastic(E, A, Lbar_1, cx1, cy1)
k1_g = k_global_2d_truss_geometric(q1, Lbar_1, cx1, cy1)

print_matrix_scaled(k1_e, scale=1, decimals=1, col_width=8)
print_matrix_scaled(k1_g, scale=1, decimals=1, col_width=8)
```

```
K = 1e+00 ×
01 |   5781.0   4335.7  -5781.0  -4335.7
02 |   4335.7   3251.8  -4335.7  -3251.8
03 |  -5781.0  -4335.7   5781.0   4335.7
04 |  -4335.7  -3251.8   4335.7   3251.8
K = 1e+00 ×
01 |      0.0     -0.0     -0.0      0.0
02 |     -0.0      0.0      0.0     -0.0
03 |     -0.0      0.0      0.0     -0.0
04 |      0.0     -0.0     -0.0      0.0
```

In [482]:

```
# -------------------------
# Member 2
# -------------------------
L2, cx2, cy2, T2 = geometry_from_nodes(nb_2, ne_2)

#Start of the problem, there is no deformation in the member yet
Lbar_2 = np.copy(L2)

v2 = Lbar_2 - L2
q2 = E * A * v2 / L2

k2_e = k_global_2d_truss_elastic(E, A, L2, cx2, cy2)
k2_g = k_global_2d_truss_geometric(q2, Lbar_2, cx2, cy2)

print_matrix_scaled(k2_e, scale=1, decimals=1, col_width=8)
print_matrix_scaled(k2_g, scale=1, decimals=1, col_width=8)
```

```
K = 1e+00 ×
01 |   5781.0  -4335.7  -5781.0   4335.7
02 |  -4335.7   3251.8   4335.7  -3251.8
03 |  -5781.0   4335.7   5781.0  -4335.7
04 |   4335.7  -3251.8  -4335.7   3251.8
K = 1e+00 ×
01 |      0.0      0.0     -0.0     -0.0
02 |      0.0      0.0     -0.0     -0.0
03 |     -0.0     -0.0      0.0      0.0
04 |     -0.0     -0.0      0.0      0.0
```

In [483]:

```
# -------------------------
# Member 3
# -------------------------
L3, cx3, cy3, T3 = geometry_from_nodes(nb_3, ne_3)

#Start of the problem, there is no deformation in the member yet
Lbar_3 = np.copy(L3)

v3 = Lbar_3 - L3
q3 = E * A * v3 / L3

k3_e = k_global_2d_truss_elastic(E, A, L3, cx3, cy3)
k3_g = k_global_2d_truss_geometric(q3, Lbar_3, cx3, cy3)

print_matrix_scaled(k3_e, scale=1, decimals=1, col_width=8)
print_matrix_scaled(k3_g, scale=1, decimals=1, col_width=8)
```

```
K = 1e+00 ×
01 |   5645.5      0.0  -5645.5     -0.0
02 |      0.0      0.0     -0.0     -0.0
03 |  -5645.5     -0.0   5645.5      0.0
04 |     -0.0     -0.0      0.0      0.0
K = 1e+00 ×
01 |      0.0     -0.0     -0.0      0.0
02 |     -0.0      0.0      0.0     -0.0
03 |     -0.0      0.0      0.0     -0.0
04 |      0.0     -0.0     -0.0      0.0
```

Part 3b - Iteration 0, Solver

In [484]:

```
# -------------------------
# Combine Element Data into Lists for Global Assembly
# -------------------------

k_list = [k1_e + k1_g, k2_e + k2_g, k3_e + k3_g]

# Note T and Qf lists are not used in this example
# just placeholders for code to work for more complex elements
T_list = [np.eye(4), np.eye(4), np.eye(4)]
Qf_list = [np.zeros(4), np.zeros(4), np.zeros(4)]
```

In [485]:

```
# -------------------------
# Assemble Global Stiffness System
# -------------------------
# This step assembles the global stiffness matrix and the global
# fixed-end force vector from all elements.
# FEFs = 0 for truss elements, but using generalized code from before

K_global, F_fef_global = assemble_global_stiffness_and_fef(
    ndof,
    k_list,
    T_list,
    Qf_list,
    map_list
    )
```

In [486]:

```
# -------------------------
# Display Global Stiffness Matrix
# -------------------------
# Print the assembled global stiffness matrix in a readable format.
# The matrix is scaled for clearer presentation in the slides.

print_matrix_scaled(K_global, scale=1, decimals=1, col_width=8)
```

```
K = 1e+00 ×
01 |  11426.5   4335.7  -5781.0  -4335.7  -5645.5      0.0
02 |   4335.7   3251.8  -4335.7  -3251.8      0.0      0.0
03 |  -5781.0  -4335.7  11562.0      0.0  -5781.0   4335.7
04 |  -4335.7  -3251.8      0.0   6503.6   4335.7  -3251.8
05 |  -5645.5      0.0  -5781.0   4335.7  11426.5  -4335.7
06 |      0.0      0.0   4335.7  -3251.8  -4335.7   3251.8
```

In [487]:

```
# -------------------------
# Partition Global System
# -------------------------
# The global stiffness equation is partitioned into free and restrained
# degrees of freedom based on the support conditions.

(
    K_ff,
    K_fr,
    K_rf,
    K_rr,
    f_f,
    f_r,
    u_r,
    f_fef_f,
    f_fef_r,
    free_dofs,
    restrained_dofs,
) = partition_system(
    K_global,
    F_global,
    u_global,
    F_fef_global,
    dof_restrained_1based
)
```

In [488]:

```
# -------------------------
# Print stiffness matrix for free DOFs
# -------------------------

print_matrix_scaled(K_ff, scale=1, decimals=1, col_width=8)
```

```
K = 1e+00 ×
01 |  11562.0      0.0  -5781.0
02 |      0.0   6503.6   4335.7
03 |  -5781.0   4335.7  11426.5
```

In [489]:

```
# -------------------------
# Solve Partitioned System
# -------------------------
# Solve for the unknown displacements at the free DOFs,
# compute the reactions at restrained DOFs, and then
# reconstruct the complete global displacement and force vectors.

rhs = f_f - f_fef_f - K_fr @ u_r
u_f = np.linalg.solve(K_ff, rhs)

u_f_iteration1 = np.copy(u_f)
print_vector_scaled(u_f_iteration1, name="u_f^(1)", scale=1, decimals=4, col_w
```

```
u_f^(1) = 1e+00 × [0.1181 -0.4650 0.2362]
```

In [490]:
```
u_global = assemble_global_displacements(
    u_f,
    u_r,
    free_dofs,
    restrained_dofs
    )

print_vector_scaled(u_global, name="u^(1)", scale=1, decimals=4, col_width=6)
```

```
u^(1) = 1e+00 × [0.0000 0.0000 0.1181 -0.4650 0.2362 0.0000]
```

Part 4a - Iteration 1, Newton-Raphson Setup

In [491]:

```
# -------------------------
# Member 1
# -------------------------
u1 = u_global[m1-1]

nb_1_new = np.array(nb_1) + np.array([u1[0], u1[1]])
ne_1_new = np.array(ne_1) + np.array([u1[2], u1[3]])

Lbar_1, cx1, cy1, T1 = geometry_from_nodes(nb_1_new, ne_1_new)

v1 = Lbar_1 - L1
q1 = E * A * v1 / L1
f1 = T1.T * q1

k1_e = k_global_2d_truss_elastic(E, A, L1, cx1, cy1)
k1_g = k_global_2d_truss_geometric(q1, Lbar_1, cx1, cy1)

print_matrix_scaled(k1_e + k1_g, scale=1, decimals=1, col_width=8)
```

```
K = 1e+00 ×
01 |   6466.2   4169.3  -6466.2  -4169.3
02 |   4169.3   2259.9  -4169.3  -2259.9
03 |  -6466.2  -4169.3   6466.2   4169.3
04 |  -4169.3  -2259.9   4169.3   2259.9
```

In [492]:

```
# -------------------------
# Member 2
# -------------------------
u2 = u_global[m2-1]

nb_2_new = np.array(nb_2) + np.array([u2[0], u2[1]])
ne_2_new = np.array(ne_2) + np.array([u2[2], u2[3]])

Lbar_2, cx2, cy2, T2 = geometry_from_nodes(nb_2_new, ne_2_new)

v2 = Lbar_2 - L2
q2 = E * A * v2 / L2
f2 = T2.T * q2

k2_e = k_global_2d_truss_elastic(E, A, L2, cx2, cy2)
k2_g = k_global_2d_truss_geometric(q2, Lbar_2, cx2, cy2)

print_matrix_scaled(k2_e + k2_g, scale=1, decimals=1, col_width=8)
```

```
K = 1e+00 ×
01 |   6466.2  -4169.3  -6466.2   4169.3
02 |  -4169.3   2259.9   4169.3  -2259.9
03 |  -6466.2   4169.3   6466.2  -4169.3
04 |   4169.3  -2259.9  -4169.3   2259.9
```

In [493]:

```
# -------------------------
# Member 3
# -------------------------
u3 = u_global[m3-1]

nb_3_new = np.array(nb_3) + np.array([u3[0], u3[1]])
ne_3_new = np.array(ne_3) + np.array([u3[2], u3[3]])

Lbar_3, cx3, cy3, T3 = geometry_from_nodes(nb_3_new, ne_3_new)

v3 = Lbar_3 - L3
q3 = E * A * v3 / L3
f3 = T3.T * q3

k3_e = k_global_2d_truss_elastic(E, A, L3, cx3, cy3)
k3_g = k_global_2d_truss_geometric(q3, Lbar_3, cx3, cy3)

print_matrix_scaled(k3_e + k3_g, scale=1, decimals=1, col_width=8)
```

```
K = 1e+00 ×
01 |   5645.5      0.0  -5645.5      0.0
02 |      0.0    161.9      0.0   -161.9
03 |  -5645.5      0.0   5645.5      0.0
04 |      0.0   -161.9      0.0    161.9
```

Part 4b - Iteration 1, Solver

In [494]:

```
# -------------------------
# Combine Element Data into Lists for Global Assembly
# -------------------------

k_list = [k1_e + k1_g, k2_e + k2_g, k3_e + k3_g]
```

In [495]:

```
# -------------------------
# Compare Internal Loads to Applied Loads at Free DOFs
# -------------------------

# This is the internal force vector
f_internal = np.zeros(ndof, dtype=float)

f_internal[np.array(m1) - 1] += np.asarray(f1).reshape(-1)
f_internal[np.array(m2) - 1] += np.asarray(f2).reshape(-1)
f_internal[np.array(m3) - 1] += np.asarray(f3).reshape(-1)

# This is the Unbalanced Force Vector at the free DOFs,
# which should be close to zero if the solution is correct.
r_iteration1 = f_f - f_internal[free_dofs]
print_vector_scaled(r_iteration1, name="r^(1)", scale=1, decimals=3, col_width
```

```
r^(1) = 1e+00 × [ 0.000 -445.021 -70.322]
```

In [496]:
```
K_global, F_fef_global = assemble_global_stiffness_and_fef(
    ndof,
    k_list,
    T_list,
    Qf_list,
    map_list
    )

(
    K_ff,
    K_fr,
    K_rf,
    K_rr,
    f_f,
    f_r,
    u_r,
    f_fef_f,
    f_fef_r,
    free_dofs,
    restrained_dofs,
) = partition_system(
    K_global,
    F_global,
    u_global,
    F_fef_global,
    dof_restrained_1based
)
```

In [497]:
```
print_matrix_scaled(K_ff, scale=1, decimals=1, col_width=8)
```

```
K = 1e+00 ×
01 |  12932.5      0.0  -6466.2
02 |      0.0   4519.7   4169.3
03 |  -6466.2   4169.3  12111.7
```

In [498]:

```
# -------------------------
# Solve Partitioned System (Unbalanced Force)
# -------------------------
# Solve for the unknown displacements at the free DOFs,
# compute the reactions at restrained DOFs, and then
# reconstruct the complete global displacement and force vectors.

rhs = r_iteration1 - f_fef_f - K_fr @ u_r
u_f = np.linalg.solve(K_ff, rhs)

du_f_iteration1 = np.copy(u_f)

print_vector_scaled(du_f_iteration1, name="du_f^(1)", scale=1, decimals=4, col
```

```
du_f^(1) = 1e+00 × [0.0338 -0.1608 0.0676]
```

In [499]:

```
# -------------------------
# Convergenece Check
# -------------------------

e = 0.01

value = np.sqrt(np.sum(du_f_iteration1**2) / np.sum(u_f_iteration1**2))
print("Convergence value:", value)

converged = value <= e
print("Converged:", converged)
```

```
Convergence value: 0.3323144606446154
Converged: False
```

Part 5a - Iteration 1, Newton-Raphson Setup

In [500]:
```
u_global = assemble_global_displacements(
    u_f_iteration1 + du_f_iteration1,
    u_r,
    free_dofs,
    restrained_dofs
    )

print_vector_scaled(u_global, name="u^(2)", scale=1, decimals=4, col_width=6)

u_f_iteration2 = u_global[free_dofs]

print_vector_scaled(u_f_iteration2, name="u_f^(2)", scale=1, decimals=4, col_w
```

```
u^(2) = 1e+00 × [0.0000 0.0000 0.1519 -0.6258 0.3038 0.0000]
u_f^(2) = 1e+00 × [0.1519 -0.6258 0.3038]
```

In [501]:

```
# -------------------------
# Member 1
# -------------------------
u1 = u_global[m1-1]

nb_1_new = np.array(nb_1) + np.array([u1[0], u1[1]])
ne_1_new = np.array(ne_1) + np.array([u1[2], u1[3]])

Lbar_1, cx1, cy1, T1 = geometry_from_nodes(nb_1_new, ne_1_new)

v1 = Lbar_1 - L1
q1 = E * A * v1 / L1
f1 = T1.T * q1

k1_e = k_global_2d_truss_elastic(E, A, L1, cx1, cy1)
k1_g = k_global_2d_truss_geometric(q1, Lbar_1, cx1, cy1)

print_matrix_scaled(k1_e + k1_g, scale=1, decimals=1, col_width=8)
```

```
K = 1e+00 ×
01 |   6705.8   4069.2  -6705.8  -4069.2
02 |   4069.2   1916.7  -4069.2  -1916.7
03 |  -6705.8  -4069.2   6705.8   4069.2
04 |  -4069.2  -1916.7   4069.2   1916.7
```

In [502]:

```
# -------------------------
# Member 2
# -------------------------
u2 = u_global[m2-1]

nb_2_new = np.array(nb_2) + np.array([u2[0], u2[1]])
ne_2_new = np.array(ne_2) + np.array([u2[2], u2[3]])

Lbar_2, cx2, cy2, T2 = geometry_from_nodes(nb_2_new, ne_2_new)

v2 = Lbar_2 - L2
q2 = E * A * v2 / L2
f2 = T2.T * q2

k2_e = k_global_2d_truss_elastic(E, A, L2, cx2, cy2)
k2_g = k_global_2d_truss_geometric(q2, Lbar_2, cx2, cy2)

print_matrix_scaled(k2_e + k2_g, scale=1, decimals=1, col_width=8)
```

```
K = 1e+00 ×
01 |   6705.8  -4069.2  -6705.8   4069.2
02 |  -4069.2   1916.7   4069.2  -1916.7
03 |  -6705.8   4069.2   6705.8  -4069.2
04 |   4069.2  -1916.7  -4069.2   1916.7
```

In [503]:

```
# -------------------------
# Member 3
# -------------------------
u3 = u_global[m3-1]

nb_3_new = np.array(nb_3) + np.array([u3[0], u3[1]])
ne_3_new = np.array(ne_3) + np.array([u3[2], u3[3]])

Lbar_3, cx3, cy3, T3 = geometry_from_nodes(nb_3_new, ne_3_new)

v3 = Lbar_3 - L3
q3 = E * A * v3 / L3
f3 = T3.T * q3

k3_e = k_global_2d_truss_elastic(E, A, L3, cx3, cy3)
k3_g = k_global_2d_truss_geometric(q3, Lbar_3, cx3, cy3)

print_matrix_scaled(k3_e + k3_g, scale=1, decimals=1, col_width=8)
```

```
K = 1e+00 ×
01 |   5645.5      0.0  -5645.5      0.0
02 |      0.0    206.5      0.0   -206.5
03 |  -5645.5      0.0   5645.5      0.0
04 |      0.0   -206.5      0.0    206.5
```

Part 5b - Iteration 1, Solver

In [504]:

```
# -------------------------
# Combine Element Data into Lists for Global Assembly
# -------------------------

k_list = [k1_e + k1_g, k2_e + k2_g, k3_e + k3_g]
```

In [505]:

```
# -------------------------
# Compare Internal Loads to Applied Loads at Free DOFs
# -------------------------

# This is the internal force vector
f_internal = np.zeros(ndof, dtype=float)

f_internal[np.array(m1) - 1] += np.asarray(f1).reshape(-1)
f_internal[np.array(m2) - 1] += np.asarray(f2).reshape(-1)
f_internal[np.array(m3) - 1] += np.asarray(f3).reshape(-1)

# This is the Unbalanced Force Vector at the free DOFs,
# which should be close to zero if the solution is correct.
r_iteration2 = f_f - f_internal[free_dofs]
print_vector_scaled(r_iteration2, name="r^(2)", scale=1, decimals=3, col_width
```

```
r^(2) = 1e+00 × [ 0.000 -52.041 -11.722]
```

In [506]:
```
K_global, F_fef_global = assemble_global_stiffness_and_fef(
    ndof,
    k_list,
    T_list,
    Qf_list,
    map_list
    )

(
    K_ff,
    K_fr,
    K_rf,
    K_rr,
    f_f,
    f_r,
    u_r,
    f_fef_f,
    f_fef_r,
    free_dofs,
    restrained_dofs,
) = partition_system(
    K_global,
    F_global,
    u_global,
    F_fef_global,
    dof_restrained_1based
)
```

In [507]:
```
print_matrix_scaled(K_ff, scale=1, decimals=1, col_width=8)
```

```
K = 1e+00 ×
01 |  13411.7      0.0  -6705.8
02 |      0.0   3833.4   4069.2
03 |  -6705.8   4069.2  12351.3
```

In [508]:

```
# -------------------------
# Solve Partitioned System (Unbalanced Force)
# -------------------------
# Solve for the unknown displacements at the free DOFs,
# compute the reactions at restrained DOFs, and then
# reconstruct the complete global displacement and force vectors.

rhs = r_iteration2 - f_fef_f - K_fr @ u_r
u_f = np.linalg.solve(K_ff, rhs)

du_f_iteration2 = np.copy(u_f)

print_vector_scaled(du_f_iteration2, name="du_f^(2)", scale=1, decimals=4, col
```

```
du_f^(2) = 1e+00 × [0.0047 -0.0234 0.0093]
```

In [509]:

```
# -------------------------
# Convergenece Check
# -------------------------

e = 0.01

value = np.sqrt(np.sum(du_f_iteration1**2) / np.sum(u_f_iteration1**2))
print("Convergence value:", value)

converged = value <= e
print("Converged:", converged)
```

```
Convergence value: 0.3323144606446154
Converged: False
```

Part 6a - Iteration 2, Newton-Raphson Setup

In [510]:
```
u_global = assemble_global_displacements(
    u_f_iteration2 + du_f_iteration2,
    u_r,
    free_dofs,
    restrained_dofs
    )

print_vector_scaled(u_global, name="u^(3)", scale=1, decimals=4, col_width=6)

u_f_iteration3 = u_global[free_dofs]

print_vector_scaled(u_f_iteration3, name="u_f^(3)", scale=1, decimals=4, col_w
```

```
u^(3) = 1e+00 × [0.0000 0.0000 0.1565 -0.6492 0.3131 0.0000]
u_f^(3) = 1e+00 × [0.1565 -0.6492 0.3131]
```